

Aviary

Metagenomic assembly, genome recovery and annotation

Aviary contributors

Released under GPL-3.0

Contents

1. From sequencing reads to microbial genomes.	4
1.1 Start with the question you have	4
1.2 Use only the stages your analysis needs.	
1.3 A complete analysis starts with one command	5
1.4 Take Aviair from installation to interpreted output.	
2. Getting started	6
2.1 Getting started	6
2.2 Installation	7
2.3 Quickstart	10
2.4 First complete analysis	12
3. Guide	14
3.1 Guide	14
3.2 Core concepts	15
3.3 Metagenome assembly	17
3.4 Genome recovery	19
3.5 Genome annotation	0
3.6 Workflow control	0
4. Worked analyses	0
4.1 Hybrid metagenome from reads to MAGs	0
4.2 Separate assembly and recovery	0
4.3 Annotate an existing genome collection	0
5. Reference	0
5.1 Reference	0
5.2 CLI	0
5.3 Configuration	0
5.4 Input reference	0
5.5 Output reference	0
6. Advanced	0
6.1 Advanced use	0
6.2 HPC & cluster submission	0
6.3 Performance and resources	0
6.4 Reproducibility	0
7. Project	0
7.1 Project information	0
7.2 Troubleshooting and FAQ	0

7.3 Citations	0
7.4 Development	0
7.5 Aviary PDF manual	0

METAGENOMICS WORKFLOW

1. From sequencing reads to microbial genomes.

Aviary assembles metagenomes, recovers metagenome-assembled genomes and coordinates taxonomic and functional annotation.

Run short-read, long-read or hybrid analyses locally or across an HPC cluster through one reproducible Snakemake workflow.

1.1 Start with the question you have

01

1.1.1 Run Aviary

Install the software, configure reference data and produce a first meaningful result.

[Quickstart](#) →

02

1.1.2 Understand the workflow

Learn how reads become assemblies, candidate genomes and annotated MAGs.

[User guide](#) →

03

1.1.3 Find an exact option

Search command syntax, accepted values, defaults and output behaviour.

[CLI reference](#) →

04

1.1.4 Resolve a problem

Diagnose real validation messages, failed rules, resource limits and run state.

[Troubleshooting](#) →

ONE WORKFLOW, SEVERAL ENTRY POINTS

1.2 Use only the stages your analysis needs.

Assemble raw reads, recover genomes from an existing assembly, annotate a MAG collection, or run the complete path. Aviary records rule logs, resource benchmarks and workflow state throughout.

[How Aviary fits together →](#)

assemble Reads → contigs

recover Assembly → MAGs

annotate MAGs → annotations

complete End-to-end analysis

1.3 A complete analysis starts with one command

```
aviary complete \
-1 sample_R1.fastq.gz \
-2 sample_R2.fastq.gz \
--output sample_aviary \
--max-threads 8 \
--n-cores 8
```

Using Aviary in research? See the [citation guide](#) for Aviary and its upstream tools. Aviary is released under GPL-3.0.

2. Getting started

2.1 Getting started

Set up Aviary, verify its reference data, and run a first analysis.

1. [Install Aviary and configure databases](#).
2. Use the [quickstart](#) for the shortest useful run.
3. Follow the [first complete analysis](#) to learn how inputs, stages, outputs and logs fit together.

For production work, also read [reproducibility](#) before starting the analysis.

2.2 Installation

Aviary targets Linux and uses isolated dependency environments for its bioinformatics tools. Bioconda is the recommended installation for users; Pixi is the supported route for development from this repository.

2.2.1 Install from Bioconda

Create a dedicated environment:

```
conda create -n aviary -c conda-forge -c bioconda aviary
conda activate aviary
```

If your Conda channels are configured globally, keep this priority:

```
channels:
- conda-forge
- bioconda
- defaults
```

Installing into a new environment avoids dependency conflicts with unrelated analysis software.

2.2.2 Verify the command

```
aviary --version
aviary complete --help
```

Before downloading large databases, use `aviary complete --help` to confirm that the installed release exposes the options used by this documentation.

2.2.3 Configure reference data

Aviary can use local data for GTDB-Tk, EggNOG-mapper, CheckM2, SingleM and Metabuli. Which resources are required depends on the workflow stages and options selected. Place shared databases outside individual run directories; on an HPC system, coordinate their location with the system administrator.

Configure existing resources explicitly:

```
aviary configure \
--gtdb-path /shared/db/gtdb/release232 \
--eggnog-db-path /shared/db/eggnog \
--checkm2-db-path /shared/db/checkm2/uniref100.K0.1.dmnd \
--singlem-metapackage-path /shared/db/singlem/package.smpkg.zb \
--metabuli-db-path /shared/db/metabuli \
--output configure_logs
```

The corresponding environment variables are:

Variable	Consumer
GTDBTK_DATA_PATH	GTDB-Tk taxonomy
EGGNOG_DATA_DIR	EggNOG functional annotation
CHECKM2DB	CheckM2 genome quality assessment
SINGLEM_METAPACKAGE_PATH	SingleM community/recovery analysis
METABULI_DB_PATH	Metabuli classification used by applicable workflows
TMPDIR	Temporary-file location

To ask Aviary to download configured resources, add `--download` followed by one or more of `gtdb`, `eggno`, `singlem`, `checkm2` and `metabuli`. Supplying `--download` without values requests all five.

```
aviary configure \
  --gtdb-path /shared/db/gtdb \
  --checkm2-db-path /shared/db/checkm2 \
  --download gtdb checkm2 \
  --output configure_logs
```

Database downloads are large and should not be duplicated per user or run. Confirm available storage and release requirements before starting them.

2.2.4 Install from source with Pixi

From a local checkout:

```
cd aviary
pixi run postinstall
pixi run aviary --version
```

This installs Aviary in editable mode. Dependency definitions live in `aviary/pixi.toml`, while `aviary/pixi.lock` pins the resolved environments. The `postinstall` task prepares both the main and development environments.

For a shared development system, database paths can be represented by symlinks in the repository's `db/` directory; `admin/set_env_vars.sh` documents the expected local names used by the activation hook.

2.2.5 Install from pip

The Python package name is `aviary-genome`, but `pip` alone does not provision the complete collection of external bioinformatics tools. Use this route only when you are deliberately managing those dependencies yourself:

```
conda env create -n aviary -f admin/environment.yml
conda activate aviary
pip install aviary-genome
```


2.2.6 Build analysis environments

After installation, Aviary can prepare its per-tool environments:

```
aviary build
```

GPU environments are optional and require compatible hardware and drivers:

```
aviary build --gpu
```

Continue with the [quickstart](#). For production clusters, see [HPC and cluster submission](#).

2.3 Quickstart

This example runs the complete short-read metagenome workflow. It assumes Aviary is installed and the required databases are configured.

2.3.1 1. Check the installation

```
aviary --version
aviary complete --help
```

2.3.2 2. Prepare paired reads

Use one forward and one reverse FASTQ file for the same sample:

```
reads/
├── sample_R1.fastq.gz
└── sample_R2.fastq.gz
```

Compressed FASTQ input is accepted. Forward and reverse files must be supplied in matching order when more than one pair is given.

2.3.3 3. Preview the workflow

```
aviary complete \
-1 reads/sample_R1.fastq.gz \
-2 reads/sample_R2.fastq.gz \
--output sample_aviary \
--max-threads 8 \
--n-cores 8 \
--dryrun
```

The dry run resolves the workflow without executing analysis tools. Remove `--dryrun` after checking the planned jobs and configured database paths.

2.3.4 4. Run

```
aviary complete \
-1 reads/sample_R1.fastq.gz \
-2 reads/sample_R2.fastq.gz \
--output sample_aviary \
--max-threads 8 \
--n-cores 8
```

`--max-threads` limits an individual tool. `--n-cores` is the total CPU capacity available to Snakemake, which may schedule several compatible jobs at once.

2.3.5 5. Inspect the results

Start with:

```
sample_aviary/  
├─ assembly/final_contigs.fasta  
├─ bins/bin_info.tsv  
├─ bins/final_bins/  
├─ benchmarks/  
└─ logs/
```

Read `bin_info.tsv` and the rest of the output layout before using recovered genomes downstream. If a rule fails, find its log in `sample_aviary/logs/` and see [troubleshooting](#).

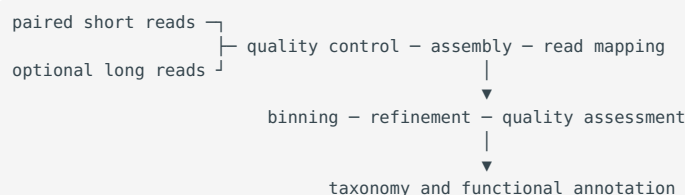
2.3.6 Next step

The [first complete analysis](#) explains how the stages behave and which controls are most important.

2.4 First complete analysis

`aviary complete` joins assembly, genome recovery and annotation into one Snakemake run. Use it when you want Aviary to carry raw reads through to a curated set of metagenome-assembled genomes (MAGs).

2.4.1 Data flow



Aviary orchestrates these stages; specialised dependencies perform the underlying assembly, binning, quality assessment and annotation algorithms.

2.4.2 Choose the input mode

Data available	Required input	Typical use
Paired short reads	<code>-1, -2</code>	Short-read metagenome assembly and recovery
Long reads	<code>--longreads, --long-read-type</code>	Long-read assembly and recovery
Both	short- and long-read options	Hybrid assembly and recovery
Existing assembly	<code>--assembly</code> plus reads for coverage	Skip de novo assembly and recover MAGs

See the [input reference](#) for pairing rules and accepted long-read type identifiers.

2.4.3 Run in a dedicated output directory

```

aviary complete \
  -1 reads/sample_R1.fastq.gz \
  -2 reads/sample_R2.fastq.gz \
  --longreads reads/sample_ont.fastq.gz \
  --long-read-type ont \
  --output sample_aviary \
  --max-threads 16 \
  --n-cores 32 \
  --max-memory 250

```

The default maximum memory value in the workflow configuration is 250 GB. Set the option to the actual hard limit available to the run; it is not a prediction of typical memory consumption.

2.4.4 Monitor and resume

Rule-specific messages are written beneath `logs/`, while Snakemake benchmark records are written beneath `benchmarks/`. Re-running the same command and output directory normally resumes from existing valid outputs. Do not delete `.snakemake/` or intermediate files while diagnosing an interrupted run.

For scheduler submission, resource caps and retries, see [HPC and cluster submission](#). For precise command options, see the [complete CLI reference](#).

3. Guide

3.1 Guide

The Aviary workflow follows sequencing data through three principal stages:

reads → assembly → genome recovery → annotation

Read [core concepts](#) first, then use the stage guide matching your analysis. The [workflow-control guide](#) explains dry runs, targeted rules, resuming and cleanup.

3.2 Core concepts

Aviary is a workflow orchestrator for metagenomic assembly and genome recovery. Understanding four distinctions makes its commands and outputs easier to use.

3.2.1 Workflow commands describe stopping points

`assemble`, `recover` and `annotate` are composable stages. `complete` runs the full path permitted by the supplied inputs, while `isolate` uses an assembly path intended for a cultured isolate rather than a mixed community.

Command	Begins with	Principal result
<code>assemble</code>	sequencing reads	assembled contigs
<code>recover</code>	assembly plus reads used for coverage	recovered MAGs
<code>annotate</code>	MAG FASTA files	taxonomy and functional annotations
<code>complete</code>	reads or an existing assembly	results through annotation
<code>cluster</code>	completed Aviary runs	dereplicated representative genomes
<code>isolate</code>	reads from a cultured isolate	isolate assembly

3.2.2 Assemblies, bins and MAGs

An assembly joins overlapping sequence evidence into contigs. Metagenomic binning then groups contigs that appear to originate from the same population. A final bin is treated as a metagenome-assembled genome (MAG), but its completeness, contamination and taxonomy remain estimates. Use `bins/bin_info.tsv` to assess each recovered genome rather than treating all FASTA files as equivalent.

3.2.3 Aviary and upstream tools

Aviary owns workflow decisions: it validates command-line input, writes the run configuration, selects Snakemake targets, passes resource limits and links final products into stable output locations. Upstream programs perform the underlying read filtering, assembly, mapping, binning, quality estimation, taxonomic classification and functional annotation. The exact tools invoked depend on command options and available data.

3.2.4 Requested resources have different scopes

`--max-threads` is the maximum made available to one tool. `--n-cores` is the total capacity Snakemake may schedule concurrently. `--local-cores` limits work kept on the coordinator node when using a cluster profile. `--max-memory` is a hard workflow cap in gigabytes, not a guarantee that every tool uses that amount.

3.2.5 Workflow state and resumability

Snakemake determines whether an output is current from its inputs, rules and metadata. A repeated Aviary command against the same output directory normally continues incomplete work. Options such as `--rerun-triggers`, `--clean` and `--unlock` deliberately change this behaviour; use them only after reading the [workflow-control guide](#).

3.2.6 Inputs and outputs

Aviary accepts FASTQ sequencing reads and FASTA assemblies or genomes. BAM files are produced internally when reads are mapped to assemblies. Final results are presented through stable directories such as `assembly/`, `bins/`, `taxonomy/` and `annotation/`; working files remain in `data/`.

Continue with the [assembly guide](#), [genome-recovery guide](#), or [annotation guide](#).

3.3 Metagenome assembly

Assembly transforms quality-controlled reads into contigs for genome recovery or other downstream analysis. Aviairy supports short-read, long-read and hybrid inputs and chooses applicable workflow rules from the data supplied.

3.3.1 Select an input strategy

- Paired short reads require matching `-1` and `-2` lists.
- Long reads require `--longreads` and a `--long-read-type` value.
- Hybrid assembly supplies both short and long reads.
- Multiple samples can be co-assembled where the selected options support it.

Input quality control happens before assembly unless it is explicitly skipped. Host references supplied through `--host-filter` are used to remove matching reads before the assembly stage.

3.3.2 Basic usage

```
aviary assemble \
-1 sample_R1.fastq.gz \
-2 sample_R2.fastq.gz \
--output assembly_run \
--max-threads 16 \
--n-cores 16
```

For a hybrid dataset, add long reads and their platform type:

```
aviary assemble \
-1 sample_R1.fastq.gz \
-2 sample_R2.fastq.gz \
--longreads sample_ont.fastq.gz \
--long-read-type ont \
--output hybrid_run
```

3.3.3 Important controls

`--min-contig-size` filters short contigs before downstream processing. Raising it reduces the fragmented sequence retained but can also discard genuine sequence from a fragmented assembly. `--long-read-assembler` selects the supported long-read assembly implementation. Quality-control thresholds affect which reads reach the assembler and should be chosen for the sequencing data, not copied mechanically between projects.

3.3.4 Result

The canonical assembly is stored in `data/final_contigs.fasta` and exposed as `assembly/final_contigs.fasta`. Reports are written under `www/`; rule logs and benchmarks are kept in `logs/` and `benchmarks/`.

See the [assemble](#) reference for every option and the [output](#) reference for the run layout.

3.4 Genome recovery

Genome recovery groups assembled contigs into candidate genomes, refines the candidate sets and evaluates the resulting MAGs. Reads are still important: mapping them back to the assembly provides coverage information used by binning and abundance calculations.

3.4.1 Basic usage

```
aviary recover \  
  --assembly assembly/final_contigs.fasta \  
  -1 sample_R1.fastq.gz \  
  -2 sample_R2.fastq.gz \  
  --output recovery_run \  
  --max-threads 16 \  
  --n-cores 32
```

3.4.2 How the stage behaves

Aviary prepares coverage inputs, invokes the enabled binners, refines candidate bins and combines evidence into a final set. Quality assessment and taxonomy then add evidence needed to interpret each MAG. A binning algorithm producing a FASTA file does not by itself establish that the genome is complete, uncontaminated or correctly classified.

3.4.3 Important controls

`--min-bin-size` excludes candidate genomes smaller than the configured number of bases from later processing. Binner selection options change which upstream algorithms contribute candidates. GPU variants require a compatible GPU and the corresponding Aviary environments; enabling a GPU flag does not make every stage GPU accelerated.

SemiBin2 multi-sample mode accepts multiple assemblies and should be used only with inputs organised for that mode. See the exact validation and accepted values in the [recover reference](#).

3.4.4 Interpreting results

Use `bins/final_bins/` as the recovered FASTA collection and `bins/bin_info.tsv` as its main summary. Retain the run's database versions and software environment because taxonomy and quality estimates can change with reference data and dependency versions.

Continue with [annotation](#), the [output reference](#), or [reproducibility guidance](#).